MODULE 27 · EXAM STRATEGY - ALL DOMAINS

Practice Exam 1 of 3

Twenty questions, mixed domains, nine seconds to answer each.

QUESTIONS 20 HOLD 9 seconds PAPER 1 of 3

admin@sthithapragna.com

How to use this paper

- **Answer within the nine seconds.** If you pause to think, you are not practising the thing that fails people.
- **Write your answers down.** A letter you said in your head is not a letter you can score.
- **Note which domain each miss came from.** That matters more than the total.
- **Do not look up anything mid-paper.** Sit it as an exam, then review afterwards.
- The mix is weighted to this course's derived domain percentages, verified across all sixty questions.

> ON THE EXAM
>
> The score is the least useful number here. Where your misses cluster is what tells you what to revise.

> A team replaces servers on every release by building a new machine image and swapping the fleet, rather than patching in place.

What is the principal advantage?

A Upgrades complete more quickly

B Upgrades require no automation

C Upgrades become less complex, because instances cannot diverge

D Upgrades can be applied without any downtime

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the principal advantage?

A Quicker

Usually the opposite. Building an image and cycling a fleet takes longer than patching in place.

B No automation

Unrelated. Immutable replacement is normally more automated, not less.

C Less complex

Correct. Every instance is built identically, so the class of problem caused by divergence stops occurring.

D No downtime

Describes what create_before_destroy aims at. Replacement itself can interrupt service.

ON THE EXAM

Domain 1. 'Quicker' is the most attractive wrong answer in this domain, and immutable is often slower.

> An engineer adds a required_providers entry with the local name dns and a version constraint, but no source argument. terraform init succeeds, and every resource type she uses is then rejected as unknown.

What happened?

A The provider requires manual installation because it is a community provider

B Terraform implied the hashicorp namespace and installed a different provider

C The version constraint excluded every published release

D The lock file pinned an incompatible version

◆ PAUSE AND DECIDE

What happened?

A **Manual installation**

True before 0.13 and false since. Official, partner and community providers all install identically.

B **Implied hashicorp namespace**

Correct. With no source, Terraform assumes the public registry, the hashicorp namespace, and the local name as the type.

C **Constraint excluded everything**

That fails init with a version error. Init succeeded, so something was installed.

D **Lock file pinned wrongly**

A lock file constrains a provider it already recorded. It cannot cause a different provider to be selected.

ON THE EXAM

Domain 2. Read the two symptoms together: init succeeded AND the resource types are unknown. That means the wrong provider, not a failed install.

> A developer adds a new module block to a configuration and runs terraform plan. Terraform stops and says the module is not installed.

What must be run, and what will it affect?

A terraform init, which touches no existing infrastructure

B terraform get -update, which also refreshes state

C terraform apply, which installs modules as part of the run

D terraform init -reconfigure, to rebuild the backend as well

◆ PAUSE AND DECIDE

What must be run, and what will it affect?

A init, affects nothing

Correct. Init handles backend, providers and modules. It creates and destroys nothing.

B get -update

terraform get does install modules, and it does not refresh state. The second half of the option is wrong.

C apply installs modules

Neither plan nor apply installs modules. That is one of init's three responsibilities.

D init -reconfigure

Discards the backend configuration for no reason. The backend did not change; a module was added.

ON THE EXAM

Domain 3. Init's three responsibilities are backend, providers and modules, and none of them reaches a running resource.

> A plan shows a managed database with the symbol -/+ and the header 'must be replaced'. One line inside the block reads 'forces replacement'.

What does this mean?

A The database will be updated in place with a brief interruption

B Terraform cannot decide and will prompt during apply

C The database will be created first, then the old one removed

D The database will be destroyed and then recreated

◆ PAUSE AND DECIDE

What does this mean?

A Updated in place

That is the tilde symbol. Minus slash plus is not an update.

B Terraform will prompt

The plan has already decided. Apply carries it out; it does not reconsider.

C Created first

That ordering only happens with create_before_destroy set, and nothing here indicates it.

D Destroyed then recreated

Correct. Minus slash plus means destroy followed by create, which for a database is a very different conversation.

ON THE EXAM

Domain 3. The 'forces replacement' line names the single attribute responsible, and is usually the thing you can change to avoid it.

> A configuration passes a list of team names directly to a resource's for_each argument. terraform plan fails immediately.

Why?

A for_each requires at least two elements

B for_each accepts a map or a set of strings, not a list

C for_each cannot be used on this resource type

D The list must be sorted before use

◆ PAUSE AND DECIDE

Why?

A **Needs two elements**

Invents a rule. One element is fine, and an empty collection is valid and produces no instances.

B **Map or set, not a list**

Correct. Wrap it in toset, which is why that function appears so often in real configuration.

C **Not available here**

for_each is a meta-argument, available on every resource type regardless of provider.

D **Must be sorted**

Sorting changes order, and a set has no order. It would not make a list acceptable.

ON THE EXAM

Domain 4. This is the first mistake almost everyone makes with for_each, and toset is the fix.

> A module author wants callers to be able to switch prevent_destroy on and off, so she sets it from an input variable.

What happens?

A Terraform errors. Lifecycle settings accept only literal values

B It works, and the variable must be marked nullable = false

C It works only when the variable has a default

D It works, but only in the root module

◆ PAUSE AND DECIDE

What happens?

A Literal values only

Correct. Lifecycle settings shape the dependency graph, so they are processed before expressions can be evaluated.

B Needs nullable = false

Invents a dependency. nullable governs whether null may be passed; it has nothing to do with lifecycle.

C Needs a default

A default changes when a value is required, not whether an expression can appear in a lifecycle block.

D Root module only

The restriction is about evaluation order, not module position. It applies everywhere.

ON THE EXAM

Domain 4. The same principle governs plan-time for_each keys and dynamic blocks not generating meta-arguments.

> A team adds a check block asserting that no storage bucket is publicly readable. Three weeks later a public bucket is deployed. The pipeline run is marked successful and the warning appears in the log.

What went wrong?

A The assertion was written incorrectly

B Check blocks run only during plan, not apply

C The check block needed a scoped data source

D A check block reports a warning and lets the operation continue

◆ PAUSE AND DECIDE

What went wrong?

A Written incorrectly

The warning is in the log, which proves the assertion evaluated false and fired exactly as designed.

B Plan only

A check block executes as the last step of both plan and apply. Neither was blocked, because warnings never block.

C Needed a data source

Would change what the check can see, not what a failure does. A better-informed warning is still a warning.

D It warns and continues

Correct. Enforcement requires a mechanism that halts: a precondition, a postcondition, or a policy.

ON THE EXAM

Domain 4. The mechanism worked perfectly. The choice of mechanism was the error.

> An engineer feeds an ephemeral random password into a database's password_wo argument, with password_wo_version set to 1. Security asks for a rotation. She reruns apply and the plan reports no changes.

Why?

A The ephemeral resource cached its value in state

B Write-only arguments can only be set at creation

C The version argument is unchanged, so no new value is accepted

D The database must be replaced to change its password

◆ PAUSE AND DECIDE

Why?

A It cached the value

Ephemeral resources persist nothing by definition, and they execute on every plan and apply. A new password genuinely was generated.

B Create-only

False. Updating is exactly what the version mechanism exists to support.

C Version unchanged

Correct. Terraform cannot see the value, so the version stored in state is the only signal it has. Increment it.

D Replace the database

Would work and rebuild a production database to change one argument. The version increment updates it in place.

ON THE EXAM

Domain 4. Ephemeral resources run every time, so without the version argument a password would appear to change on every apply.

> A shared module reads a startup script that ships alongside its Terraform files. It works when the author tests it from the module's own directory, and fails for every consumer.

What is the most likely cause?

A Modules may only contain .tf files

B The module uses path.root, which resolves to the caller's directory

C Consumers must run terraform get before the file is available

D File contents must be passed in as a variable

◆ PAUSE AND DECIDE

What is the most likely cause?

A Only .tf files

Modules routinely ship scripts, templates and policy documents. Nothing restricts the directory.

B path.root

Correct. In the author's own directory path.root and path.module are identical, so the bug hides exactly where it would be caught.

C Run terraform get

Init already installs modules, and the module was clearly installed since Terraform reached the point of reading the file.

D Pass it in

Would work and pushes a detail onto every consumer. Modules can read their own files perfectly well.

ON THE EXAM

Domain 4. path.module for anything a module ships. Testing from the module's own directory cannot catch this class of bug.

> A configuration in the same repository references a sibling directory as a module source, written as modules/vpc with no prefix. terraform init fails, reporting that it cannot find the module in the registry.

What is wrong?

A. A local path must begin with ./ or ../

B. Local modules require a version argument

C. The module must be published before it can be used

D. Sibling directories cannot be used as modules

◆ PAUSE AND DECIDE

What is wrong?

A Needs ./ or ../

Correct. That prefix is how Terraform distinguishes a filesystem path from a registry address. Without it, the string is a registry address.

B Needs a version

Backwards. The version argument works only with registry sources and would be rejected on a local path.

C Must be published

Local modules are the normal way to factor out repeated code within one repository.

D Siblings not allowed

A relative path going up and across is entirely valid, written with dot dot slash.

ON THE EXAM

Domain 5. The error message names the registry, which tells you Terraform read the string as a registry address rather than a path.

> A regulated team commits the dependency lock file and pins providers with pessimistic constraints. They apply the same pessimistic constraints to their registry modules. Two environments then deploy different module versions.

What should change?

A Commit the lock file in both environments

B Run terraform init -upgrade before every apply

C Switch the modules to Git sources with no ref

D Use exact version constraints for the modules

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should change?

A Commit the lock file

They already do, and it would not help. The lock file records provider selections only.

B -upgrade every time

The opposite of what they need. It re-resolves to the newest matching release, maximising divergence.

C Git with no ref

Removes even the constraint they had. An unpinned Git source takes whatever is on the default branch.

D Exact module versions

Correct. Modules are never recorded in the lock file, so the constraint is the entire pinning mechanism.

ON THE EXAM

Domain 5. A pessimistic constraint is safe for providers because the lock file pins underneath it. Modules have nothing underneath.

> A team is provisioning state storage on AWS. Their three-year-old runbook specifies a versioned, encrypted S3 bucket plus a DynamoDB table for locking.

Which part is out of date, and what replaces it?

A Versioning, replaced by state snapshots

B Encryption, now applied automatically by the backend

C DynamoDB locking, replaced by use_lockfile

D Nothing. The runbook is current

◆ PAUSE AND DECIDE

Which part is out of date, and what replaces it?

A **Versioning**

Still recommended, and the real protection against losing state.

B **Encryption**

Still configured deliberately, and still recommended alongside restricted access.

C **DynamoDB, use_lockfile**

Correct. dynamodb_table is deprecated. Native S3 locking writes a .tflock object beside the state file.

D **Nothing**

The honest trap. It still works, which is exactly why nobody has revisited it. Working and current differ.

ON THE EXAM

Domain 6. This is the most reliably out-of-date pattern in Terraform writing, including material published recently.

> A team moves state to a bucket in a different account. They edit the backend block, run init, misread the prompt, and rerun with -reconfigure. The next plan proposes to create every resource. Nothing has been destroyed.

What happened?

A -reconfigure discarded the old configuration without copying state

B The resources were destroyed during the migration

C The new bucket needs versioning enabled first

D Apply will detect and adopt the existing resources

◆ PAUSE AND DECIDE

What happened?

A -reconfigure copied nothing

Correct. It starts the new backend fresh. -migrate-state is the flag that brings existing state across.

B Destroyed

The question says nothing was destroyed, and migration never destroys infrastructure. Only apply changes anything.

C Versioning first

Versioning protects against losing state. It is not a precondition for reading it.

D Apply adopts them

The most dangerous option. Terraform never adopts existing objects by matching them; it would build duplicates.

ON THE EXAM

Domain 6. The infrastructure is fine. What was lost is Terraform's knowledge of it, which is why the next apply looks reasonable.

> An engineer runs terraform import for a manually created bucket, using the address she intends to use. The command reports success. The next plan proposes to destroy the bucket.

Why?

A The import silently failed and left a partial state entry

B No resource block exists at that address

C The bucket must be tagged before Terraform recognises it

D terraform import requires a second invocation to complete

◆ PAUSE AND DECIDE

Why?

A Silent failure

It reported success and succeeded. State now holds a correct binding, which is why the plan can see something to destroy.

B No resource block

Correct. Import writes state and never writes configuration. A binding with nothing declared reads as a resource you deleted.

C Tagging

Terraform identifies objects by what state records. Tags were tried as an identity mechanism and abandoned.

D Two invocations

One completes the import. What is missing is the other half of the job, which is yours to write.

ON THE EXAM

Domain 7. Import creates a binding in state. The configuration is still your job, and this is the most common import mistake.

> An engineer is asked to capture verbose logs. She exports TF_LOG_PATH pointing at a file and runs terraform apply. The apply completes and the file exists, completely empty.

What is wrong?

A The path must be relative to the working directory

B Logs are written only when a command fails

C TF_LOG_PATH was superseded by a command-line flag

D TF_LOG must also be set, or nothing is logged

◆ PAUSE AND DECIDE

What is wrong?

A **Must be relative**

Absolute paths work. The file was created where she asked, which proves Terraform read the variable.

B **Only on failure**

Logging is level-based, not outcome-based. A successful TRACE run produces a great deal of output.

C **Superseded by a flag**

There is no such flag. Terraform's logging is configured entirely through environment variables.

D **TF_LOG is the switch**

Correct. Even when TF_LOG_PATH is set, TF_LOG must be set for any logging to be enabled.

ON THE EXAM

Domain 7. The empty file is the clue. Terraform read the path and created the file, then had no logging enabled to write into it.

> An organisation enforces Sentinel policies on every workspace. One team switched their workspace's execution mode to Local for easier debugging. An audit later finds policy-violating resources deployed from that workspace, with no policy failures recorded.

What is the explanation?

A The policies were misconfigured and passed everything

B Sentinel applies only to VCS-driven workspaces

C Local execution mode disables the features that depend on remote operations

D The team used -auto-approve, which bypasses policy checks

◆ PAUSE AND DECIDE

What is the explanation?

A Misconfigured

They worked everywhere else. One workspace differing points at a workspace setting, not at the policies.

B VCS-driven only

Policies apply across workflows. Execution mode is what matters, not what triggered the run.

C Local disables them

Correct. HashiCorp lists policy enforcement, cost estimation and notifications as requiring remote execution.

D -auto-approve

Skips the interactive confirmation, not the policy stage. A failing policy would still stop the run.

ON THE EXAM

Domain 8. Nothing errored and nothing was recorded. The policies were not failing; they were never evaluated.

> A security team writes a policy for a legal data residency requirement. Deployments outside approved regions must never be permitted, under any circumstances, by anyone.

Which enforcement level applies?

A Advisory

B Hard-mandatory

C Soft-mandatory

D Advisory, with a run task to block the apply

◆ PAUSE AND DECIDE

Which enforcement level applies?

A Advisory

Logs a warning and lets the run continue. The deployment would proceed.

B Hard-mandatory

Correct. It stops the run and nobody can override it, which is what 'never, by anyone' requires.

C Soft-mandatory

Stops the run and permits an override by someone with the right permission. The requirement forbids any override.

D Advisory plus a run task

A run task could gate it, and it is an elaborate route to something one enforcement level does natively.

ON THE EXAM

Domain 8. Advisory warns, soft-mandatory can be overridden, hard-mandatory cannot. The requirement states its level in plain English.

> A configuration must deploy resources into two regions of the same cloud account, in a single plan, with the dependency between them visible.

How should this be configured?

A. A second provider block with an alias, and the provider meta-argument on the second region's resources

B. A second entry in required_providers, one per region

C. A CLI workspace per region

D. Two root modules, applied separately

◆ PAUSE AND DECIDE

How should this be configured?

A Alias plus meta-argument

Correct. One plugin, two configurations, both regions in one plan with a real dependency between them.

B Two required_providers entries

That block holds one entry per local name and says which plugin to install. Region is not part of a plugin's identity.

C A workspace per region

Only one workspace is active at a time, so the two regions could never appear in one plan.

D Two root modules

Works, and splits the plan in two. The requirement is a single plan with the dependency visible.

ON THE EXAM

Domain 2. 'Single plan, dependency visible' is the sentence that eliminates the two options which would otherwise work.

> A team applies with -target daily so they only touch the service they are working on. Over six months, three incidents were traced to resources nobody targeted. They ask how to make targeting safer.

What should they be told?

A Target the parent module instead, so dependencies are included

B Run an untargeted apply weekly to catch stragglers

C Stop routine targeting and split the configuration, sharing values via data sources

D Increase parallelism so full applies are fast enough

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should they be told?

A **Target the module**

Widens the selection and is still targeting. The undetected drift HashiCorp warns about is unchanged.

B **Weekly full apply**

Reduces the window without closing it, and means a weekly apply carrying six days of unreviewed change.

C **Split the configuration**

Correct, and the alternative HashiCorp names: smaller configurations applied independently, connected by data sources.

D **Raise parallelism**

Treats a design problem as throughput. Provider rate limits usually bite first.

ON THE EXAM

Domain 3. The question asks how to make targeting safer, and the documented answer is to stop needing it.

> A change board approves a plan on Monday. It is saved with -out. On Thursday an unrelated team applies an incident fix to the same configuration. On Friday the saved plan is applied.

What should the team expect?

A Terraform will prompt, because the plan is no longer current

B Terraform will regenerate the plan, since saved plans expire

C Terraform will merge Monday's plan with Thursday's changes

D The apply may fail or produce an unintended result

◆ PAUSE AND DECIDE

What should the team expect?

A It will prompt

Apply never prompts for a saved plan. The approval is treated as having happened when the plan was generated.

B Plans expire

Invents a lifetime. A plan file is stale the moment anything changes, and it never announces that.

C It merges

Terraform has no merge concept. A plan is a fixed set of actions computed against one moment.

D May fail or misfire

Correct. HashiCorp warns that intervening changes can make the final effect differ, and advises re-checking before applying.

ON THE EXAM

Domain 3. A saved plan guarantees that what was approved is what runs. It cannot guarantee that what runs still makes sense.

Scoring paper 1

- **The answer key, in order:** C B A D B A D C B A D C A B D C B A C D.
- **Now group your misses by domain**, using the label at the bottom of each walkthrough.
- **Two misses in one domain matters more than six spread evenly.** A cluster is a gap; a scatter is noise.
- **Do not re-sit this paper.** You would be testing recall of these questions, not of Terraform.
- Papers two and three cover different topics, and one question in each deliberately inverts one from this paper.

> ON THE EXAM
>
> Revisit the module for any domain where you missed more than one, then take paper two. The score itself tells you very little.